

Gradient Descent vs. Genetic Algorithm for Urban Sound Classification

Custom Smart Adaptive Systems · UPC 2025/26

Leonie Greber

June 15, 2026

Outline

Dataset

Architecture

Mathematical Background

Parameter Tuning

Results

Dataset: UrbanSound8K

- ▶ **8 674 audio clips**, 10 urban-sound classes
- ▶ Raw audio reduced to **34 numerical features** per clip
- ▶ Features extracted once; training operates on tabular data

10 classes:

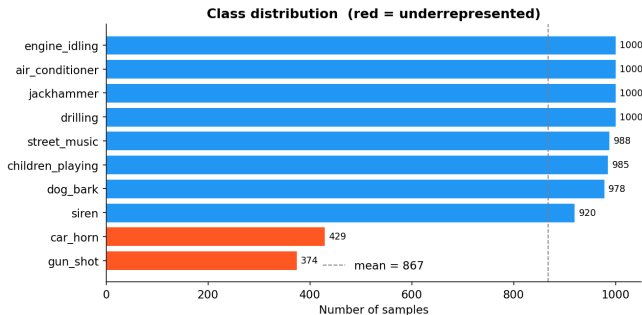
air conditioner, car horn, children playing,
dog bark, drilling, engine idling,
gun shot, jackhammer, siren, street music

Split (stratified):

70% train · 10% val · 20% test

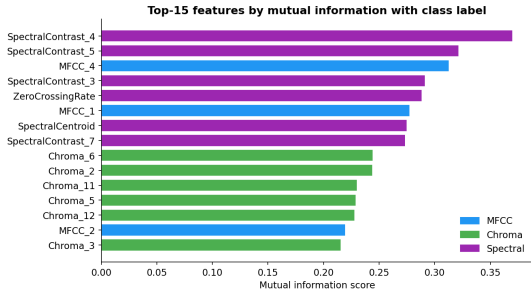
Feature group	Dimensions
MFCC (mel-frequency cepstral coefficients)	13
Chroma	12
Spectral contrast	7
Zero-crossing rate	1
Spectral centroid	1
Total	34

Dataset: Class Distribution

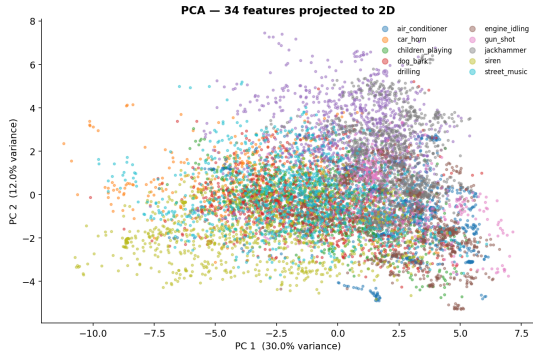


- ▶ Most classes: $\approx 1\,000$ samples
- ▶ **car_horn** (≈ 429) and **gun_shot** (≈ 374) are underrepresented
- ▶ Stratified split ensures proportional representation in every fold
- ▶ These two classes will show lower F1 (by construction, not model failure)

Dataset: Feature Importance and Class Separability



MFCC features dominate mutual information



Classes overlap in 2D; non-linear boundaries needed

- ▶ PCA retains $\approx 42\%$ of variance in 2 components; substantial structure remains in higher dims
- ▶ A non-linear model (MLP with ReLU) is therefore justified

MLP Architecture

Forward pass:

$$z^{(1)} = W_1x + b_1, \quad h = \text{ReLU}(z^{(1)})$$

$$z^{(2)} = W_2h + b_2 \quad (\text{logits})$$

Input(34) $\rightarrow$ Linear(34 $\rightarrow$ 20) $\rightarrow$ ReLU $\rightarrow$ Linear(20 $\rightarrow$ 10)

Parameter count:

Layer	Formula	Count
Linear(34 $\rightarrow$ 20)	$34 \times 20 + 20$	700
Linear(20 $\rightarrow$ 10)	$20 \times 10 + 10$	210
Total		910

- ▶ **Loss:** CrossEntropyLoss (log-softmax + NLL)
- ▶ Forward returns *raw logits*; no explicit softmax layer
- ▶ **Same architecture** for both GD and GA; only the weight-update mechanism differs
- ▶ GA treats the MLP as a black box: no gradients required

Why Hidden Size = 20? (Bias-Variance Constraint)

Rule of thumb: 5–10 training samples per parameter

$$\frac{6,072 \text{ train samples}}{910 \text{ parameters}} \approx 6.7 \Rightarrow \text{within the safe zone}$$

Hidden	Params	Samples/param	Risk
10	460	13.2	underfitting
20	910	6.7	good
30	1,360	4.5	overfitting
40	1,810	3.4	overfitting
50	2,260	2.7	overfitting

- ▶ Larger hidden layer can fit train data better but *generalises worse* on a dataset of this size
- ▶ This choice is *confirmed empirically* by the parameter sweep in Section 4 (train–val gap widens for $h > 20$)
- ▶ Weight decay ($\lambda = 10^{-4}$) adds L2 regularisation as a second line of defence

GD: Loss Function and Gradient

Cross-entropy loss (multi-class, N samples, $C = 10$ classes):

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y_i}(x_i; \theta) \quad (1)$$

$$\hat{p}_k(x) = \frac{e^{z_k(x)}}{\sum_{j=1}^C e^{z_j(x)}} \quad (\text{softmax}) \quad (2)$$

Vanilla gradient descent update:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (3)$$

- ▶ $\nabla_{\theta} \mathcal{L}$ is computed via **backpropagation** (chain rule through both linear layers)
- ▶ Mini-batch size 64 — trade-off between gradient noise (regularisation) and convergence speed
- ▶ With weight decay λ : effective loss is $\mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2$

GD: Adam Optimizer

Adam maintains *per-parameter* running estimates of gradient mean and variance:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction}) \quad (6)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (7)$$

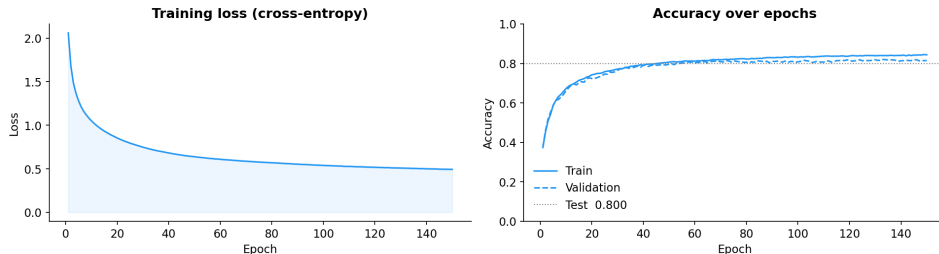
Parameters used:

$$\eta = 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$$
$$\epsilon = 10^{-8}, \quad \lambda_{\text{wd}} = 10^{-4}$$

- ▶ $\sqrt{\hat{v}_t}$ normalises the step: large-gradient params take smaller steps
- ▶ Bias correction avoids small initial estimates biasing early updates

GD: Training Dynamics

Gradient Descent (Adam)



- ▶ Loss (log scale) falls near-linearly: confirms exponential decay, plateauing after epoch 80
- ▶ Train and val accuracy track closely, confirming hidden= 20 does not overfit
- ▶ Log scale makes the convergence rate visible; the slope flattens after epoch 80

GA: Operators

The GA searches the 910-dimensional weight space *without gradients*.

Fitness (improved):

$$f(\theta) = -\mathcal{L}_{\text{val}}(\theta) \quad (8)$$

Higher fitness = lower validation loss;
continuous signal vs. coarse accuracy.

BLX- α crossover ($\alpha = 0.5$):

$$o_i \sim \mathcal{U}[l_i, u_i] \quad (9)$$

$$l_i = \min(p_i, q_i) - \alpha|p_i - q_i|,$$
$$u_i = \max(p_i, q_i) + \alpha|p_i - q_i|$$

Allows offspring outside parent range.

Gaussian mutation ($p_m = 0.05$ per gene):

$$\theta_i \leftarrow \theta_i + \mathcal{N}(0, \sigma^2), \sigma = 0.05 \quad (10)$$

Per-gene probability: ≈ 46 of 910 weights
perturbed per individual.

Selection: Tournament of size 5: sample 5
individuals, keep the best. Repeat $N - k$ times
to fill the next generation.

GA: Algorithm per Generation

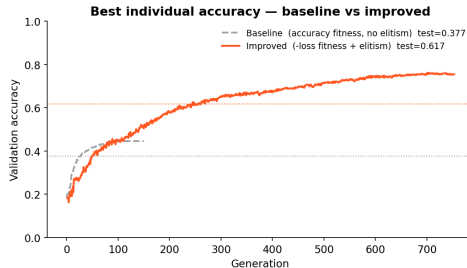
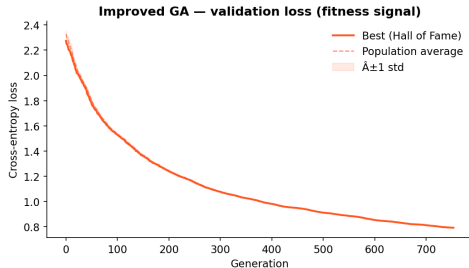
One GA generation (pseudocode)

1. Evaluate fitness for all individuals with invalid fitness
2. Select $N - k$ offspring via tournament selection
3. For each consecutive pair: apply BLX- α crossover with probability $c = 0.7$
4. For each offspring: apply Gaussian mutation with probability $p_m = 0.05$
5. Preserve top- k **elite** individuals unchanged (elitism, $k = 5$)
6. Replace population: offspring $\cup$ elites (N individuals total)
7. Update Hall of Fame (best individual ever seen across all generations)

- ▶ **Baseline GA** used raw accuracy fitness and eaSimple (no elitism); test acc 0.377
- ▶ **Key improvements:** fitness = $-\mathcal{L}_{\text{val}} + \text{elitism} \Rightarrow$ test acc 0.621
- ▶ **Early stopping:** patience of 50 generations without improvement; avoids wasted compute

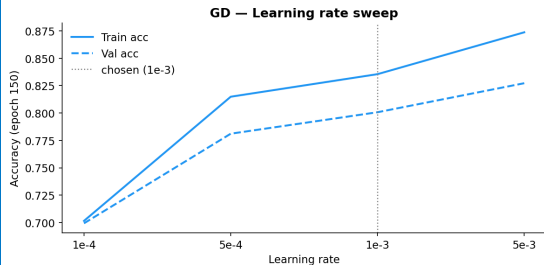
GA: Convergence (Improved)

Genetic Algorithm — convergence

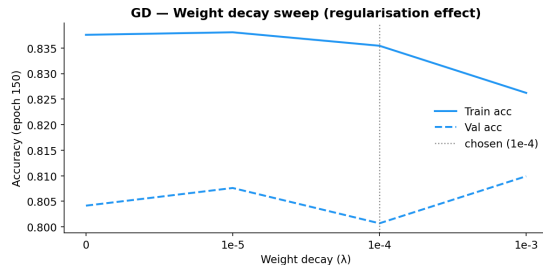


- ▶ Left: validation loss of best individual (HoF) vs population average; gap shows elites pull ahead
- ▶ Right: accuracy of improved GA far exceeds baseline across all generations
- ▶ Population std (shaded) gradually narrows; diversity decreases as elites dominate

GD: Hyperparameter Sweeps (Learning Rate and Weight Decay)



Chosen: $\eta = 10^{-3}$

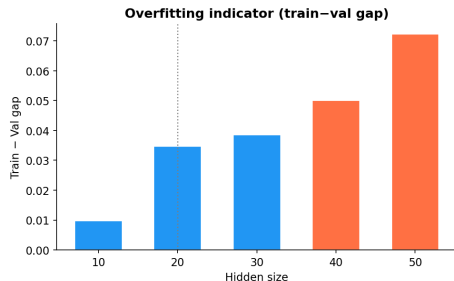
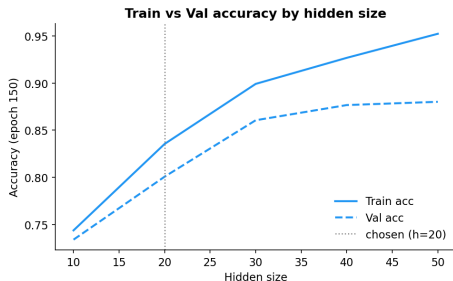


Chosen: $\lambda = 10^{-4}$

All sweeps fix other parameters at chosen values; 150 epochs; seed fixed. Hidden-size sweep: see next slide.

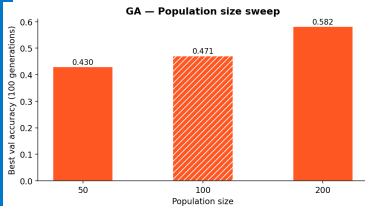
Bias-Variance Tradeoff: Hidden Size in Detail

GD — Hidden size sweep (bias-variance tradeoff)

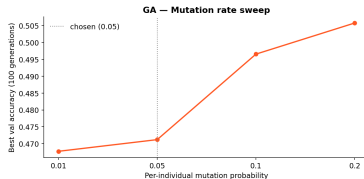


- ▶ $h = 10$: both train and val acc are modest; **underfitting** (insufficient capacity)
- ▶ $h = 20$: small train-val gap, good val acc; **well-calibrated**
- ▶ $h = 30-50$: train acc rises, val acc plateaus/drops; **overfitting**, gap widens
- ▶ Weight decay compresses the gap but also limits peak accuracy at high h

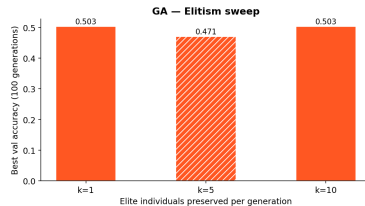
GA: Hyperparameter Sweeps



Chosen: $N = 100$



Chosen: $p_m = 0.05$

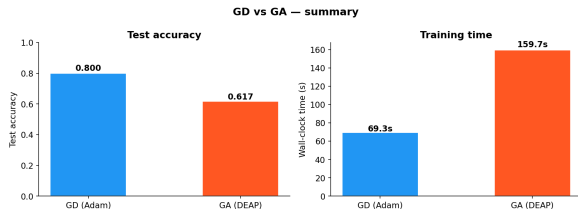


Chosen: $k = 5$

Sweeps use 100 generations each; $c = 0.7$ (crossover prob) fixed. Hatched bar = chosen value.

- ▶ Larger populations explore more but with diminishing returns and higher per-generation cost
- ▶ Low mutation ($p_m = 0.01$) causes premature convergence; high ($p_m = 0.2$) destroys good solutions
- ▶ Elitism $k = 5$ reliably outperforms $k = 1$: a single elite can be lost to crossover; five provide a stable backbone

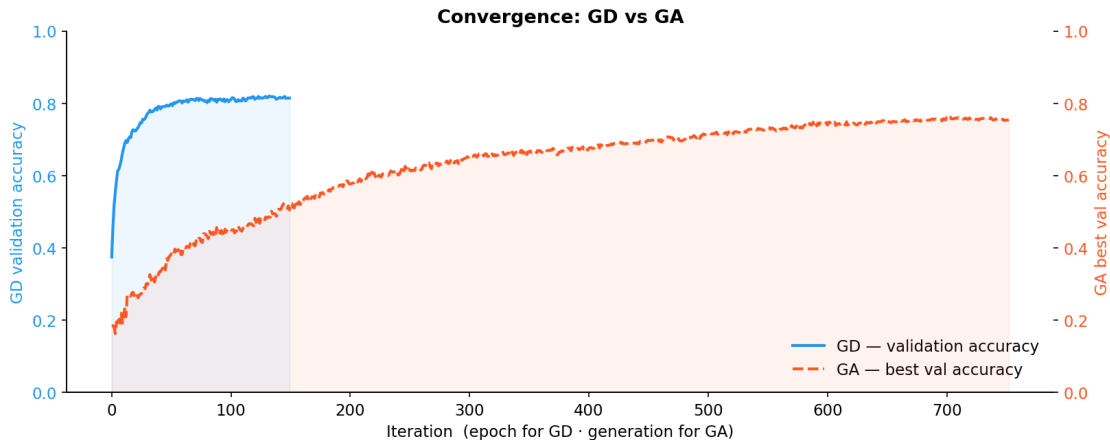
Results: Accuracy and Training Time



	GD	GA
Test accuracy	0.811	0.621
Training time	≈ 37 s	≈ 124 s
Macro F1	$\approx$ 0.81	≈ 0.60
Stopped at	150 epochs	gen ≈ 799

- ▶ GD is $\approx 30\%$ more accurate and $3.4\times$ faster
- ▶ GA's gradient-free search struggles in 910 dimensions
- ▶ Elitism + $-\mathcal{L}$ fitness roughly doubled GA accuracy over baseline ($0.377 \rightarrow 0.621$)

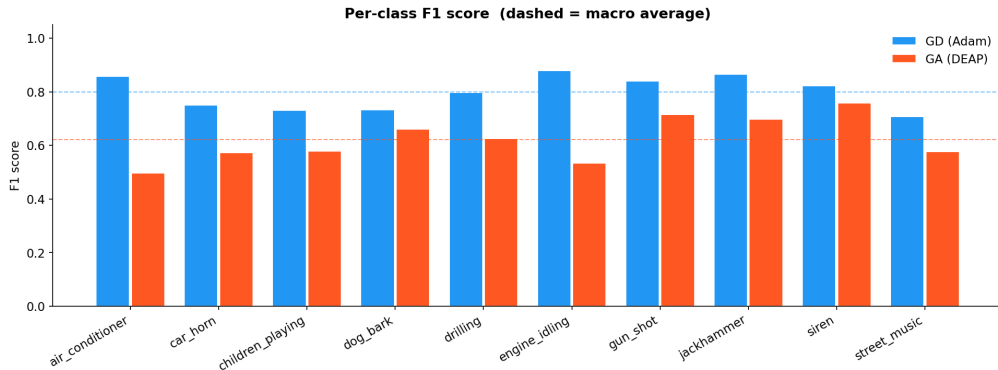
Results: Convergence



► GD converges in < 50 epochs; val accuracy

► GA improves slowly across ≈ 800

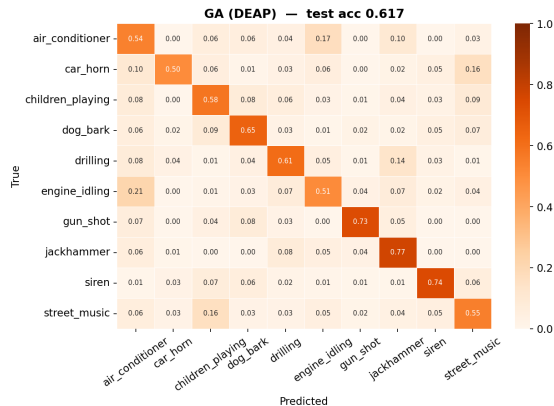
Results: Per-class F1 Score



- ▶ GD dominates on all 10 classes; both methods struggle most with *gun_shot* and *car_horn*
- ▶ Low F1 on those two classes reflects the class imbalance, not model failure

Results: Confusion Matrices

Confusion matrices (row-normalised)



- ▶ GA confusion matrix shows more off-diagonal mass; less discriminative across all classes
- ▶ GD confuses acoustically similar pairs (e.g. jackhammer/drilling); GA confuses almost everything

Conclusion

Key findings:

- ▶ GD (Adam) clearly dominates: 81.1% vs 62.1% accuracy, 3× faster
- ▶ Continuous $-\mathcal{L}$ fitness + elitism are *essential* for GA (baseline: 37.7%)
- ▶ Hidden = 20 is empirically confirmed as the optimal capacity for this dataset
- ▶ Both methods agree: *gun_shot* and *car_horn* are hardest (class imbalance)

When does GA make sense?

- ▶ Non-differentiable objectives (hardware-in-the-loop, discrete search spaces)
- ▶ Black-box optimisation where gradient computation is impossible

Future work:

- ▶ CMA-ES: adapts the mutation covariance matrix, far more efficient in high dimensions
- ▶ Hybrid: GD warm-start → GA fine-tune for non-smooth loss surfaces
- ▶ Neuroevolution of architectures (NEAT): evolve topology, not just weights